LAPORAN PRAKTIKUM PEKAN 6

PEMROGRAMAN BERORIENTASI OBJEK

ABSTRACTION

Disusun Oleh:

Syasya Halwa Gazwani

(2511531018)

Dosen Pengampu:

Afdhal Dinilhak, M.Kom.

Asisten Praktikum:

Ezza Addini

(2311532001)

Link Repository: https://github.com/syasyahalwa22/PrakPBO_B_2511531018

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Assalamu'alaikum warahmatullahi wabarakatuh,

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Pemrograman Berorientasi Objek, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW. yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini. Penulis mengucapkan terima kasih kepada bapak Nurfiah,S.ST., M.Kom. selaku dosen mata kuliah Pemrograman Berorientasi Objek yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan mata kuliah yang penulis tekuni. Penulis juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga penulis dapat menyelesaikan Laporan Praktikum ini.

Padang, November 2026

Penulis

BAB I PENDAHULUAN

1.1 Latar Belakang

Pemrograman Berorientasi Objek merupakan salah sastu konsep penting dalam pemrograman yang digunakan untuk membuat program berdasarkan objek tertentu. Dengan ini, suatu permasalahan dapat diselesaikan secara terstruktur, karena dapat menyelesaikan masalah tersebut dengan mengelompokkan objek-objek tersebut. Konsep dasar PBO meliputi class, object, constructor, atribut, dan method. Class berfungsi sebagai rancangan untuk membuat object, sedangkan object merupakan bentuk nyata dari class. Atribut digunakan untuk menyimpan data atau keadaan object, sementara method digunakan untuk mendefinisikan tindakan atau perilaku yang dapat dilakukan oleh object. Constructor digunakan untuk memberikan nilai awal ketika sebuah object dibuat.

1.2 Tujuan

1.2.1 Meningkatkan pemahaman tentang dasar-dasar bahasa Java dalam PBO.

1.2.2 Dapat membuat sebuah program dengan menggunakan atribut-atribut yang akan digunakan.

1.2.3 Melatih kemampuan logika pemrograman.

1.3 Manfaat Praktikum

1.3.1 Menambah pemahaman tentang dasar-dasar bahasa Java dalam PBO

1.3.2 Melatih kemampuan membuat program menggunakan bahasa Java.

1.3.3 Memahami proses pembuatan program pada Java.

BAB II PEMBAHASAN

DASAR TEORI

Pemrograman Berorientasi Objek (*Object-Oriented Programming*/OOP), yaitu pendekatan yang menyusun program sebagai kumpulan objek berisi data dan perilaku, dengan empat pilar. Enkapsulasi menyembunyikan atribut dengan private dan *getter* (misalnya CustomerContact yang *immutable*), pewarisan memungkinkan EmailChannel, SmsChannel, dan WhatsAppChannel memakai send() dari AbstractChannel, abstraksi diwujudkan lewat antarmuka NotificationChannel dan kelas abstrak AbstractChannel, sedangkan polimorfisme membuat NotificationService dapat memanggil send() pada berbagai kanal melalui tipe antarmuka tanpa mengetahui jenis aslinya. Semuanya dirangkai dengan pola desain *Template Method*, yaitu send() yang bersifat final menetapkan urutan baku pengiriman, sementara langkah resolveRecipient, prepare, dan deliver diimplementasikan oleh tiap kanal.

LANGKAH KERJA

1. Buka VS Code dan tekan Ctrl+Shift+P, lalu ketika **Java: Create Java Project...,** setelah itu pilih **No build tools**
2. Pilih folder induk tempat proyek disimpan (D:\praktikum-oop\p6).
3. Beri nama proyek abstraction-2511531018, lalu VS Code akan membuka proyek baru di jendela tersebut.
4. VS Code akan membuat struktur seperti:

```
abstraction-2401010057/
├── .vscode/settings.json   <- tells VS Code that the source folder is src
├── lib/                    <- external .jar files (not used today)
├── src/App.java            <- sample program
└── README.md
```

5. Membuat folder package pada panel Explorer dengan klik kanan src lalu **New Java Package**. Ketik berikut satu persatu.

```
shop/model
shop/shipping
shop/discount
shop/payment
shop/service
shop/app
```

LATIHAN PRAKTIKUM

1. Class Product.java

```java
package shop.model;

public abstract class Product {
    private final String sku;
    private final String name;
    private final long price; // in rupiah

    protected Product(String sku, String name, long price) {
        if (sku == null || sku.isBlank()) {
            throw new IllegalArgumentException("SKU must not be blank");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        if (price < 0) {
            throw new IllegalArgumentException("Price must not be negative");
        }
        this.sku = sku;
        this.name = name;
        this.price = price;
    }

    public String getSku() { return sku; }
    public String getName() { return name;}
    public long getPrice() { return price; }

    /** Price the customer actually pays. Subclasses may override it. */
    public long getFinalPrice() { return price; }

    public abstract String category();

    public String getInfo() {
        return "[" + category() + "] " + name + " - Rp" + getFinalPrice();
    }
}
```

Gambar 1 Kode program Class Product.java

```java
package shop.model;

public abstract class Product {
    private final String sku;
    private final String name;
    private final long price;
```

Package shop.model untuk menempatkan class pada folder shop.model.

abstract class berarti Product tidak bisa dibuat objeknya secara langsung.

Private final yaitu hanya bisa diakses dari dalam class ini saja dan nilainya hanya bisa diisi sekali di constructor (bersifat immutable yang tidak bisa diubah setelah dibuat). Sku untuk menyimpan kode unik produk, name untuk nama produk, dan price untuk harga dasar produk dalam rupiah.

```
protected Product(String sku, String name, long price) {
    if (sku == null || sku.isBlank()) {
        throw new IllegalArgumentException("SKU must not be blank");
    }
    if (name == null || name.isBlank()) {
        throw new IllegalArgumentException("Name must not be blank");
    }
    if (price < 0) {
        throw new IllegalArgumentException("Price must not be negative");
```

Constructor membuat objek produk baru sekaligus memvalidasi datanya. Modifier protected constructor hanya bisa dipanggil oleh subclass lewat super(...), bukan langsung dari luar. Jika sku dan name bernilai null atau kosong, program melempar IllegalArgumentException. Pengecekan null ditaruh paling depan supaya isBlank() tidak dipanggil pada null. Pada price, jika harga negatif ditolak karena tidak masuk akal.

```
this.sku = sku;
this.name = name;
this.price = price;
```

Setelah semua validasi lolos, nilai parameter disimpan ke field milik objek. Kata this dipakai untuk membedakan field class dengan parameter yang namanya sama.

```
public String getSku() { return sku; }
public String getName() { return name;}
public long getPrice() { return price; }
```

Method getsku untuk mengambil nilai sku dengan return String, field sku bersifat private, sehingga class lain hanya bisa membacanya lewat getter ini. Method name untuk mengambil nama produk dengan return String. Method getPrice untuk mengambil harga dasar produk dengan return long.

```
public long getFinalPrice() { return price; }

public abstract String category();

public String getInfo() {
    return "[" + category() + "] " + name + " - Rp" + getFinalPrice();
}
```

getFinalPrice untuk mengambil harga yang benar-benar dibayar pelanggan dengan return long. Method abstract category() untuk menyatakan kategori produk dengan return String. Setiap subclass wajib mengimplementasikannya. Alasannya, Product tidak tahu produk itu termasuk kategori apa, sehingga ditentukan oleh subclass masing-masing. Method getInfo memanggil category() dan getFinalPrice(), bukan field price langsung. Jadi kalau subclass meng-override getFinalPrice() (misalnya diskon), hasil getInfo() otomatis ikut menyesuaikan. Ini menunjukkan polimorfisme dan pola Template Method.

2. Class PhysicalProduct.java

```java
package shop.model;

import shop.shipping.Shippable;

public class PhysicalProduct extends Product implements Shippable {
    private final int weightGrams;


    public PhysicalProduct(String sku, String name, long price, int weightGrams) {
        super(sku, name, price);

        if (weightGrams <= 0) {
            throw new IllegalArgumentException("Weight must be greater than zero");
        }

        this.weightGrams = weightGrams;
    }

    @Override
    public int getWeightGrams() {
        return weightGrams;
    }

    @Override
    public String category() {
        return "Fisik";
    }
}
```

Gambar 2 Kode program Class PhysicalProduct.java

```java
package shop.model;

import shop.shipping.Shippable;

public class PhysicalProduct extends Product implements Shippable {
    private final int weightGrams;
```

extends Product berarti class ini mewarisi semua isi Product. Implements Shippable berarti class ini berjanji memiliki method yang ditetapkan interface

Shippable, yaitu getWeightGrams(). WeightGrams adalah atribut tambahan khusus produk fisik: berat dalam gram. Atribut ini private final, jadi tidak bisa diubah setelah objek dibuat.

```java
public PhysicalProduct(String sku, String name, long price, int weightGrams) {
    super(sku, name, price);

    if (weightGrams <= 0) {
        throw new IllegalArgumentException("Weight must be greater than zero");
    }

    this.weightGrams = weightGrams;
}
```

super(sku, name, price); memanggil constructor Product untuk menyimpan dan memvalidasi sku, name, dan price. Pemanggilan super(...) harus berada di baris pertama constructor. Validasi berat, jika weightGrams <= 0, dilempar IllegalArgumentException. Jadi berat harus lebih dari 0. this.weightGrams = weightGrams; menyimpan berat ke field.

```java
    @Override
    public int getWeightGrams() {
        return weightGrams;
    }

    @Override
    public String category() {
        return "Fisik";
    }
}
```

Method getWeightGrams() untuk mengambil berat produk dalam gram dengan return int. @Override menandakan method ini adalah implementasi dari method getWeightGrams() milik interface Shippable. Method category untuk mengisi method abstrak category() dari Product.

3. Class DigitalProduct.java

```java
package shop.model;

import shop.shipping.Shippable;

public class PhysicalProduct extends Product implements Shippable {
    private final int weightGrams;


    public PhysicalProduct(String sku, String name, long price, int weightGrams) {
        super(sku, name, price);

        if (weightGrams <= 0) {
            throw new IllegalArgumentException("Weight must be greater than zero");
        }

        this.weightGrams = weightGrams;
    }

    @Override
    public int getWeightGrams() {
        return weightGrams;
    }

    @Override
    public String category() {
        return "Fisik";
    }
}
```

Gambar 3 Kode program Class DigitalProduct.java

```java
package shop.model;

import shop.discount.Discountable;

public class DigitalProduct extends Product implements Discountable {
    private final String downloadUrl;
    private static final int DIGITAL_DISCOUNT_PERCENT = 10;
```

Fungsi class ini untuk produk digital yang diunduh lewat link dan mendapat diskon, extends Product untuk mewarisi isi Product. implements Discountable berjanji memiliki method diskon dari interface Discountable. downloadUrl: link unduhan produk. DIGITAL_DISCOUNT_PERCENT = 10: konstanta besar diskon produk digital, yaitu 10%. static final artinya nilainya satu untuk semua objek dan tidak bisa diubah.

```java
public DigitalProduct(String sku, String name, long price, String downloadUrl) {
    super(sku, name, price);

    if (downloadUrl == null || downloadUrl.isBlank()) {
        throw new IllegalArgumentException("Download URL must not be blank");
    }

    this.downloadUrl = downloadUrl;
}
```

super(sku, name, price); meneruskan tiga data pertama ke Product. Validasi downloadUrl: tidak boleh null atau kosong, kalau tidak dilempar IllegalArgumentException. Pengecekan null didahulukan agar isBlank() tidak error. this.downloadUrl = downloadUrl; menyimpan link ke field.

```
public String getDownloadUrl() {
    return downloadUrl;
}

@Override
public int discountPercent() {
    return DIGITAL_DISCOUNT_PERCENT;
}
```

Method getDownloadURL() untuk mengambil link unduhan. Method discountPercent untuk memberi tahu besar diskon produk dalam persen.

```
@Override
public long getFinalPrice() {
    return applyDiscount(getPrice());
}

@Override
public String category() {
    return "Digital";
}
```

Method getFinalPrice untuk menghitung harga akhir setelah diskon. Method ini meng override getFinalPrice() milik Product, yang tadinya hanya mengembalikan price. getPrice() mengambil harga dasar, lalu applyDiscount(...) menerapkan diskon. Method category untuk mengisi method abstrak category().

4. Class ShippingService.java

```
package shop.shipping;

public abstract class ShippingService {
    private final String name;
    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        this.name = name;
    }

    public String getName() { return name; }

    /** Rate in rupiah per started kilogram. */
    protected abstract long ratePerKg();

    /** Weight is rounded up: 700 g -> 1 kg, 1001 g -> 2 kg. */
    public final long calculateCost(int weightGrams) {
        if (weightGrams <= 0) {
            return 0;
        }
        long kilograms = (weightGrams + 999L) / 1000;
        return kilograms * ratePerKg();
    }
}
```

Gambar 4 Kode program Class ShippingService.java

```java
package shop.shipping;

public abstract class ShippingService {
    private final String name;
    protected ShippingService(String name) {
```

Fungsi class ini sebagai kerangka dasar layanan pengiriman. Class ini abstrak, jadi tidak bisa dibuat langsung. Diturunkan menjadi jenis pengiriman tertentu. Atribut name adalah nama layanan. Protected hanya subclass yang bisa memanggilnya lewat super(...).

```java
    if (name == null || name.isBlank()) {
        throw new IllegalArgumentException("Name must not be blank");
    }
    this.name = name;
}
```

Ada validasi yang dimana nama tidak boleh null atau kosong. Jika valid, nama disimpan ke field.

```java
public String getName() { return name; }

/** Rate in rupiah per started kilogram. */
protected abstract long ratePerKg();
```

Method getName() untuk mengambil nama layanan pengirimanan. Method ratePerKg untuk menyatakan tarif per kilogram.

```java
public final long calculateCost(int weightGrams) {
    if (weightGrams <= 0) {
        return 0;
    }
    long kilograms = (weightGrams + 999L) / 1000;
    return kilograms * ratePerKg();
}
```

Method calculateCost untuk menghtiung ongkos kirim berdasarkan berat dengan parameter weightGrams yaitu berat barang dalam gram. Jika berat <= 0, ongkos 0. (weightGrams + 999L) / 1000 membulatkan berat ke atas ke kilogram. Karena pembagian int/long di Java membuang sisa, menambah 999 dulu membuat sisa berapa pun naik ke kilogram berikutnya. Suffix L membuat penjumlahan memakai long. kilograms * ratePerKg() mengalikan jumlah kilogram dengan tarif.

5. Class RegularShipping.java

```java
package shop.shipping;

public class RegularShipping extends ShippingService {
    public RegularShipping() {
        super(name: "Reguler");
    }

    @Override
    protected long ratePerKg() {
        return 10_000;
    }
}
```

Gambar 5 Kode program class RegularShipping.java

```java
package shop.shipping;

public class RegularShipping extends ShippingService {
    public RegularShipping() {
        super(name: "Reguler");
    }
```

Class RegularShipping berfungsi untuk layanan pengiriman reguler. Class ini tidak memiliki parameter dan mewarisi class ShippingService. Class ini langsung memanggil super("Reguler") untuk mengisi nama layanan. Jadi pengguna cukup menulis new RegularShipping().

```java
    @Override
    protected long ratePerKg() {
        return 10_000;
    }
}
```

Method ratePerKg() mengisi method abstrak dari ShippingService. Dengan return 10_000 (per kg).

6. Class ExpressShipping.java

```java
package shop.shipping;

public class ExpressShipping extends ShippingService {
    public ExpressShipping() {
        super(name: "Ekspres");
    }

    @Override
    protected long ratePerKg() {
        return 18_000;
    }

}
```

Gambar 6 Kode program class ExpressShipping.java

```
package shop.shipping;

public class ExpressShipping extends ShippingService {
    public ExpressShipping() {
        super(name: "Ekspres");
    }
```

Fungsi class ExpressShipping digunakan untuk layanan pengiriman ekspres (lebih cepat, lebih mahal). Constructor ExpressShipping() ini tanpa parameter, memanggil super("Ekspres") untuk mengisi nama.

```
    @Override
    protected long ratePerKg() {
        return 18_000;
    }
}
```

Method ratePerKg() return atau mengembalikan 18_000 (per kg).

7. Interface Shippable.java

```
package shop.shipping;

public interface Shippable {
    int getWeightGrams();

}
```

Gambar 7 Kode program Interface Shippable.java

Fungsi interface ini sebagai “kontrak” untuk benda yang dapat dikirim, yaitu harus punya berat. Interface hanya berisi daftar method tanpa isi dan class yang implements interface ini wajib mengisi semuanya. getWeightGrams() adalah satu-satunya method, mengembalikan berat dalam gram (int). Method di interface otomatis public abstract, jadi tidak perlu ditulis. Di kode ini, PhysicalProduct yang mengimplementasikannya, DigitalProduct tidak, karena produk digital tidak perlu dikirim.

8. Interface PaymentMethod.java

```java
package shop.payment;

public interface PaymentMethod {
    String getName();

    boolean pay(long amount);
}
```

Gambar 8 Kode program Interface PaymentMethod.java

Fungsi interface ini digunakan sebagai kontrak untuk metode pembayaran, setiap class yang mengimplementasikannya wajib memiliki dua method ini. Method getName untuk mengambil nama metode pembayaran. Method pay(long amount) untuk melakukan pembayaran dan hasil nya akan true jika berhasil, false jika gagal.

9. Class EwalletPayment.java

```java
package shop.payment;

public class EWalletPayment implements RefundablePayment {
    private final String provider;
    private long balance;

    public EWalletPayment(String provider, long balance) {
        if (provider == null || provider.isBlank()) {
            throw new IllegalArgumentException("Provider must not be blank");
        }

        if (balance < 0) {
            throw new IllegalArgumentException("Balance must not be negative");
        }

        this.provider = provider;
        this.balance = balance;
    }

    @Override
    public String getName() {
        return "E-Wallet " + provider;
    }

    public long getBalance() {
        return balance;
    }
```

```java
package shop.payment;

public class EWalletPayment implements RefundablePayment {
    private final String provider;
    private long balance;

    public EWalletPayment(String provider, long balance) {
        if (provider == null || provider.isBlank()) {
            throw new IllegalArgumentException("Provider must not be blank");
        }

        if (balance < 0) {
            throw new IllegalArgumentException("Balance must not be negative");
        }

        this.provider = provider;
        this.balance = balance;
    }

    @Override
    public String getName() {
        return "E-Wallet " + provider;
    }

    public long getBalance() {
        return balance;
    }
```

Gambar 9 Kode program Class EwalletPayment.java

```java
package shop.payment;

public class EWalletPayment implements RefundablePayment {
    private final String provider;
    private long balance;
```

Berfungsi sebagai pembayaran lewat dompet digital yang punya saldo, dan bisa di refund dengan parameter provider sebagai nama penyedia dompet dan balance yaitu saldo saat ini (tidak final karena saldo berubah tiap bayar atau refund).

```java
public EWalletPayment(String provider, long balance) {
    if (provider == null || provider.isBlank()) {
        throw new IllegalArgumentException("Provider must not be blank");
    }

    if (balance < 0) {
        throw new IllegalArgumentException("Balance must not be negative");
    }

    this.provider = provider;
    this.balance = balance;
```

Constructor EwalletPayment dengan parameter provider dan balance. Melakuka perulangan provider tidak boleh null atau kosong dan balance tidak boleh negatif.

```java
@Override
public String getName() {
    return "E-Wallet " + provider;
}

public long getBalance() {
    return balance;
}
```

Getter untuk membaca saldo. Method ini khusus milik EWalletPayment, tidak ada di interface. Mengembalikan name dan balance.

```java
@Override
public boolean pay(long amount) {
    if (amount <= 0 || amount > balance) {
        return false;
    }

    balance -= amount;
    return true;
}
```

Berfungsi membayar dengan memotong saldo. Gagal (false) jika amount nol/negatif, atau lebih besar dari saldo. Berhasil (true) jika valid, saldo dikurangi amount.

```java
@Override
public boolean refund(long amount) {
    if (amount <= 0) {
        return false;
    }

    balance += amount;
    return true;
}
```

Berfungsi mengmbalikan dana ke saldo. Gagal jika amount <= 0. Jika valid, saldo ditambah amount.

10. Class BankTransferPayment.java

```java
package shop.payment;

public class BankTransferPayment implements PaymentMethod {
    private final String bankName;

    public BankTransferPayment(String bankName) {
        this.bankName = bankName;
    }

    @Override
    public String getName() {
        return "Transfer " + bankName;
    }

    @Override
    public boolean pay(long amount) {
        return amount > 0;
    }

}
```

Gambar 10 Kode program Class BankTransferPayment.java

```java
package shop.payment;

public class BankTransferPayment implements PaymentMethod {
    private final String bankName;
```

Berfungsi sebagai pembayaran lewat transfer bank yang hanya mengimplementasikan PayentMethod, bukan RefundablePayment, sehingga tidak bisa di refund.

```java
public BankTransferPayment(String bankName) {
    this.bankName = bankName;
}
```

Constructor BankTransferPayment(String bankName) menyimpan nama bank. Tidak ada validasi, jadi null akan lolos dan menghasilkan "Transfer null"

```java
@Override
public String getName() {
    return "Transfer " + bankName;
}

@Override
public boolean pay(long amount) {
    return amount > 0;
}
```

getName(): mengembalikan "Transfer " + bankName, misalnya "Transfer Bank Nagari". pay(long amount): mengembalikan true selama amount > 0. Tidak ada saldo yang dicek karena ini simulasi sederhana.

11. Class CheckoutService.java

```java
package shop.service;

import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;
import shop.payment.RefundablePayment;

public class CheckoutService {
    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shippable) {
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }
```


```java
package shop.service;

import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;
import shop.payment.RefundablePayment;

public class CheckoutService {
    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shippable) {
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }
```


Gambar 11 Kode program Class CheckoutService.java

```java
package shop.service;

import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;
import shop.payment.RefundablePayment;
```

Berfungsi sebagai pusat logika belanja untuk menghitung subtotal, berat, lalu memproses pembayaran dan refund.

```
public class CheckoutService {
    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }
```

subtotal(List<Product> items)

Berfungsi untuk menjumlahkan harga akhir semua produk di keranjang. Yang dijumlahkan adalah getFinalPrice(), bukan getPrice(), jadi diskon produk digital otomatis ikut terhitung.

```
public int totalWeight(List<Product> items) {
    int grams = 0;
    for (Product product : items) {
        if (product instanceof Shippable shippable) {
            grams += shippable.getWeightGrams();
        }
    }
    return grams;
}
```

totalWeight(List<Product> items)

Berfungsi menjumlahkan berat semua barang yang bisa dikirim. product instanceof Shippable shippable mengecek apakah produk itu Shippable. Jika ya, sekaligus dibuat variabel shippable yang siap dipakai. Produk digital dilewati karena tidak Shippable, jadi tidak menambah berat.

```
public long total(List<Product> items, ShippingService shipping) {
    return subtotal(items) + shipping.calculateCost(totalWeight(items));
}
```

total(List<Product> items, ShippingService shipping)

Total bayar = subtotal + ongkos kirim. Parameter bertipe ShippingService, sehingga bisa diisi RegularShipping maupun ExpressShipping.

```
public boolean checkout(List<Product> items, ShippingService shipping, PaymentMethod payment) {
    if (items == null || items.isEmpty()) {
        return false;
    }
    return payment.pay(total(items, shipping));
}
```

Berfungsi untuk memproses pembayaran satu keranjang. Jika keranjang null atau kosong, langsung false. Lalu, memanggil payment.pay(total). Parameter payment bertipe interface PaymentMethod, jadi method ini bisa menerima e-wallet, transfer, kartu kredit, bahkan voucher, tanpa tahu class aslinya.

```java
public boolean refund(RefundablePayment payment, long amount) {
    return payment.refund(amount);
}
```

refund(RefundablePayment payment, long amount)

Meneruskan permintaan refund ke metode pembayaran. Parameternya bertipe RefundablePayment, jadi kompiler sendiri yang mencegah refund ke metode yang tidak mendukungnya.

12. Class ShopApp.java

```java
package shop.app;

import java.util.List;
import shop.model.DigitalProduct;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.RegularShipping;
import shop.shipping.ShippingService;
import shop.payment.BankTransferPayment;
import shop.payment.EWalletPayment;
import shop.payment.CreditCardPayment;
import shop.payment.PaymentMethod;
import shop.model.Address;
import shop.model.Order;

public class ShopApp {
    Run | Debug
    public static void main(String[] args) {
        Product coffee = new PhysicalProduct(sku: "KOPI-001", name: "Kopi Arabika Gayo 250g", price: 85_000, weightGrams: 300);
        Product rendang = new PhysicalProduct(sku: "RDG-002", name: "Rendang Kemasan 250g", price: 120_000, weightGrams: 400);
        Product ebook = new DigitalProduct(sku: "EBK-003", name: "E-book Resep Kue Nusantara", price: 50_000,
            downloadUrl: "https://toko-umkm.example/unduh/ebk-003");

        List<Product> cart = List.of(coffee, rendang, ebook);
        System.out.println("== Katalog ==");

        for (Product product : cart) {
            System.out.println(product.getInfo());
        }
```

```java
    CheckoutService checkout = new CheckoutService();

    ShippingService regular = new RegularShipping();
    ShippingService express = new ExpressShipping();

    System.out.println("\n== Ringkasan ==");

    System.out.println("Subtotal : Rp" + checkout.subtotal(cart));
    System.out.println("Berat kiriman : " + checkout.totalWeight(cart) + " g");
    System.out.println("Total Reguler : Rp" + checkout.total(cart, regular));
    System.out.println("Total Ekspres : Rp" + checkout.total(cart, express));

    EWalletPayment wallet = new EWalletPayment(provider: "DompetKu", balance: 300_000);
    PaymentMethod transfer = new BankTransferPayment(bankName: "Bank Nagari");
    System.out.println("\n== Pembayaran ==");
    System.out.println(wallet.getName() + " berhasil? " + checkout.checkout(cart, regular, wallet));
    System.out.println("Sisa saldo: Rp" + wallet.getBalance());
    System.out.println(transfer.getName() + " berhasil? " + checkout.checkout(cart, express, transfer));

    CreditCardPayment card = new CreditCardPayment(cardHolder: "Siti Rahma", creditLimit: 1_000_000);
    System.out.println(card.getName() + " berhasil? " + checkout.checkout(cart, express, card));
    System.out.println("Cicilan 3 bulan: Rp" + card.monthlyInstallment(checkout.total(cart, express), months: 3));

    System.out.println("\n== Refund e-book ==");
    System.out.println("Refund e-wallet berhasil? " + checkout.refund(wallet, ebook.getFinalPrice()));
    System.out.println("Saldo setelah refund: Rp" + wallet.getBalance());
    // checkout.refund(transfer, 1_000); // compile error: BankTransferPayment is not RefundablePayment

    Address address = new Address(street: "Jl. Moh Hatta No. 10", city: "Padang", postalCode: "25121");
    Order order = new Order(orderId: "ORD-0001", address);

    order.addLine(coffee, quantity: 2);
    order.addLine(ebook, quantity: 1);
```

```
        Order.Receipt receipt = order.new Receipt();
        System.out.println("\n" + receipt.toText());

        PaymentMethod voucher = new PaymentMethod() {
            private long remaining = 50_000;

            @Override
            public String getName() {
                return "Voucher Promo Hari Ini";
            }

            @Override
            public boolean pay(long amount) {
                if (amount <= 0 || amount > remaining) {
                    return false;
                }

                remaining -= amount;
                return true;
            }
        };
        System.out.println("\n" + voucher.getName() + " untuk e-book berhasil? "
            + checkout.checkout(List.of(ebook), regular, voucher));
    }
}
```

Gambar 12 Kode program Class ShopApp.java

```
package shop.app;

import java.util.List;
import shop.model.DigitalProduct;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.RegularShipping;
import shop.shipping.ShippingService;
import shop.payment.BankTransferPayment;
import shop.payment.EWalletPayment;
import shop.payment.CreditCardPayment;
import shop.payment.PaymentMethod;
import shop.model.Address;
import shop.model.Order;
```

Class yang berisi main yang mendemonstrasikan semua class.

```
public class ShopApp {
    Run | Debug
    public static void main(String[] args) {
        Product coffee = new PhysicalProduct(sku: "KOPI-001", name: "Kopi Arabika Gayo 250g", price: 85_000, weightGrams: 300
        Product rendang = new PhysicalProduct(sku: "RDG-002", name: "Rendang Kemasan 250g", price: 120_000, weightGrams: 400)
        Product ebook = new DigitalProduct(sku: "EBK-003", name: "E-book Resep Kue Nusantara", price: 50_000,
            downloadUrl: "https://toko-umkm.example/unduh/ebk-003");
```

Variabelnya bertipe Product, tetapi objeknya PhysicalProduct dan DigitalProduct (polimorfisme). List.of(...) membuat daftar yang tidak bisa diubah.

```
List<Product> cart = List.of(coffee, rendang, ebook);
System.out.println("== Katalog ==");

for (Product product : cart) {
    System.out.println(product.getInfo());
}
```

Berfungsi untuk menampilkan katalog.

```
CheckoutService checkout = new CheckoutService();

ShippingService regular = new RegularShipping();
ShippingService express = new ExpressShipping();
```

Menyimpan nama-nama variabel.

```
System.out.println("\n== Ringkasan ==");

System.out.println("Subtotal : Rp" + checkout.subtotal(cart));
System.out.println("Berat kiriman : " + checkout.totalWeight(cart) + " g");
System.out.println("Total Reguler : Rp" + checkout.total(cart, regular));
System.out.println("Total Ekspres : Rp" + checkout.total(cart, express));
```

Mencetak subtotal, berat barang yang dikirim, total dengan pengiriman regular, dan total dengan pengiriman ekspress.

```
EWalletPayment wallet = new EWalletPayment(provider: "DompetKu", balance: 300_000);
PaymentMethod transfer = new BankTransferPayment(bankName: "Bank Nagari");
System.out.println("\n== Pembayaran ==");
System.out.println(wallet.getName() + " berhasil? " + checkout.checkout(cart, regular, wallet));
System.out.println("Sisa saldo: Rp" + wallet.getBalance());
System.out.println(transfer.getName() + " berhasil? " + checkout.checkout(cart, express, transfer));
```

Menyimpan variabel-varibael dengan nama yang baru. Pada EwalletPayment yaitu (“Dompetku”, 300_000) dan BankTransferPayment(“Bank Nagari”). Lalu program mencetak seluruh program-programnya.

```
CreditCardPayment card = new CreditCardPayment(cardHolder: "Siti Rahma", creditLimit: 1_000_000);
System.out.println(card.getName() + " berhasil? " + checkout.checkout(cart, express, card));
System.out.println("Cicilan 3 bulan: Rp" + card.monthlyInstallment(checkout.total(cart, express), months: 3));

System.out.println("\n== Refund e-book ==");
System.out.println("Refund e-wallet berhasil? " + checkout.refund(wallet, ebook.getFinalPrice()));
System.out.println("Saldo setelah refund: Rp" + wallet.getBalance());
```

Mencetak seluruh program-program.

```
Address address = new Address(street: "Jl. Moh Hatta No. 10", city: "Padang", postalCode: "25121");
Order order = new Order(orderId: "ORD-0001", address);

order.addLine(coffee, quantity: 2);
order.addLine(ebook, quantity: 1);

Order.Receipt receipt = order.new Receipt();
System.out.println("\n" + receipt.toText());
```

Membuat alamat, membuat pesanan ORD-0001, menambah dua baris pesanan (kopi 2 buah, e-book 1 buah), lalu mencetak struk. Sintaks order.new Receipt() menandakan Receipt adalah inner class (class di dalam Order), sehingga objeknya harus dibuat dari sebuah objek Order.

```
PaymentMethod voucher = new PaymentMethod() {
    private long remaining = 50_000;

    @Override
    public String getName() {
        return "Voucher Promo Hari Ini";
    }
```

```
    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > remaining) {
            return false;
        }

        remaining -= amount;
        return true;
    }
};
System.out.println("\n" + voucher.getName() + " untuk e-book berhasil? "
    + checkout.checkout(List.of(ebook), regular, voucher));
```

Class tanpa nama yang langsung mengimplementasikan PaymentMethod di tempat, berguna untuk implementasi sekali pakai. Mengambil nilai name dan mengembalikan “Voucher promo hari ini”. Voucher punya sisa Rp50.000. Total belanja e-book saja: 45.000 + ongkos kirim 0 (e-book tidak berbobot) = Rp45.000. Karena 45.000 ≤ 50.000, hasilnya true.

13. Interface Discountable.java

```
package shop.discount;

public interface Discountable {
    int discountPercent();
    default long applyDiscount(long price) {
        return price - percentOf(price, safePercent());
    }

    static long percentOf(long amount, int percent) {
        return amount * percent / 100;
    }

    private int safePercent() {
        int percent = discountPercent();
        if (percent < 0) {
            return 0;
        }
        return Math.min(percent, 100);
    }

}
```

Gambar 13 Kode program Interface Discountable.java

```
package shop.discount;

public interface Discountable {
    int discountPercent();
    default long applyDiscount(long price) {
        return price - percentOf(price, safePercent());
    }
```

Fungsi interface sebagai kontrak untuk produk yang bisa diberi diskon, sekaligus menyediakan rumus dikonnya. Class yang implements Discountable wajib mengisinya dengan besar diskon dalam persen. Default method untuk menghitung harga setelah diskon dengan parameter price (harga sebelum

diskon). Default membuat method ini punya isi dalam interface, sehingga class implementasi tidak wajib menulis ulang.
`static long percentOf(long amount, int percent) { return amount * percent / 100; }` Berfungsi untuk menghtung berapa rupiah nilai persen dari amount. Method static di interface dipanggil lewat nama interface, bukan lewat objek.
`private int safePercent() { int percent = discountPercent(); if (percent < 0) { return 0; } return Math.min(percent, 100); }` Berfungsi sebagai pengaman agar diskon selalu berada di rentang 0-100. Jika discountPercent() bernilai negatif, dianggap 0. Jika lebih dari 100, dipotong menjadi 100 lewat Math.min.

14. Interface RefundablePayment

```java
package shop.payment;

public interface RefundablePayment extends PaymentMethod {
    boolean refund(long amount);

}
```

Gambar 14 Kode program Interface RefundablePayment.java

Interface ini mewarisi getName() dan pay(), lalu menambahkan satu method baru. refund(long amount) berguna untuk mengembalikan dana senilai amount, return true jika berhasil, false jika gagal. Tujuannya memisahkan mana pembayaran yang bisa di-refund dan mana yang tidak. Transfer bank sengaja tidak mengimplementasikannya.

15. Interface InstallmentPayment

```java
package shop.payment;

public interface InstallmentPayment extends PaymentMethod {
    int maxInstallmentMonths();

    default long monthlyInstallment(long amount, int months) {
        if (months < 1 || months > maxInstallmentMonths()) {
            throw new IllegalArgumentException(
                "Months must be between 1 and " + maxInstallmentMonths());
            }
            return (amount + months - 1) / months;
        }
    }
```

Gambar 15 Kode program Interface InstallmentPayment.java

```java
package shop.payment;

public interface InstallmentPayment extends PaymentMethod {
    int maxInstallmentMonths();
```

Berfungsi sebagai kontrak untuk pembayaran yang mendukung ciciclan. maxInstallMonths() untuk mengembalikan jumlah bulan cicilan maksimum dan diisi oleh class implementasi.

```java
default long monthlyInstallment(long amount, int months) {
    if (months < 1 || months > maxInstallmentMonths()) {
        throw new IllegalArgumentException(
            "Months must be between 1 and " + maxInstallmentMonths());
        }
        return (amount + months - 1) / months;
    }
```

monthlyInstallmentMonths() untuk menghitung cicilan per bulan dengan parameter amount (total yang dicicil) dan months (jumlah bulan) yang mengembalikan nilai long cicilan per bulan. Dihitung dengan (amount + months – 1) / months. Jika months kurang dari 1 atau lebih dari batas maksimum, dilempar IllegalArgumentException.

16. Class CreditCardPayment.java

```java
package shop.payment;

public class CreditCardPayment implements RefundablePayment, InstallmentPayment {
    private final String cardHolder;
    private final long creditLimit;
    private long usedCredit;

    public CreditCardPayment(String cardHolder, long creditLimit) {
        this.cardHolder = cardHolder;
        this.creditLimit = creditLimit;
        this.usedCredit = 0;
    }

    @Override
    public String getName() {
        return "Kartu Kredit (" + cardHolder + ")";
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > creditLimit - usedCredit) {
            return false;
        }

        usedCredit += amount;
        return true;
    }
```

```java
    @Override public boolean refund(long amount) {
        if (amount <= 0 || amount > usedCredit) {
            return false;
        }

        usedCredit -= amount;
        return true;
    }

    @Override
    public int maxInstallmentMonths() {
        return 12;
    }

    public long getUsedCredit() {
        return usedCredit;
    }
}
```

Gambar 16 Kode program Class CreditCardPayment.java

```java
package shop.payment;

public class CreditCardPayment implements RefundablePayment, InstallmentPayment {
    private final String cardHolder;
    private final long creditLimit;
    private long usedCredit;
```

Class ini berguna untuk pembayaran kartu kredit dengan limit, bisa di-refund, dan bisa dicicil. implements RefundablePayment, InstallmentPayment menunjukkan sebuah class bisa mengimplementasikan beberapa interface

sekaligus. Kedua interface sama-sama turunan PaymentMethod, sehingga getName() dan pay() cukup ditulis sekali. cardHolder yitu nama pemegang kartu, creditLimit yaitu batas kredit (final), dan usedCredit yaitu kredit yang sudah terpakai (berubah-ubah).

```
public CreditCardPayment(String cardHolder, long creditLimit) {
    this.cardHolder = cardHolder;
    this.creditLimit = creditLimit;
    this.usedCredit = 0;
```

Constructor CreditPayment dengan parameter cardHolder dan creditLimit. usedCredit otomatis diisi 0 karena kartu baru belum dipakai. Seperti BankTransferPayment, constructor ini belum memvalidasi input.

```
@Override
public String getName() {
    return "Kartu Kredit (" + cardHolder + ")";
}
```

Mengambil dan mengembalikan "Kartu Kredit (" + cardHolder + ")".

```
@Override
public boolean pay(long amount) {
    if (amount <= 0 || amount > creditLimit - usedCredit) {
        return false;
    }

    usedCredit += amount;
    return true;
```

creditLimit - usedCredit adalah sisa limit. Gagal jika amount tidak valid atau melebihi sisa limit. Jika berhasil, usedCredit bertambah.

```
@Override public boolean refund(long amount) {
    if (amount <= 0 || amount > usedCredit) {
        return false;
    }

    usedCredit -= amount;
    return true;
```

Refund(long amount)

Refund hanya boleh sampai sebesar kredit yang terpakai. Berbeda dengan e-wallet yang menerima refund tanpa batas atas.

```
@Override
public int maxInstallmentMonths() {
    return 12;
}

public long getUsedCredit() {
    return usedCredit;
}
```

maxInstallmentMonths()

Mengembalikan 12, artinya cicilan maksimal 12 bulan. Method monthlyInstallment() diwarisi otomatis dari InstallmentPayment. getUsedCredit() Getter untuk melihat kredit yang sudah terpakai.

17. Class Address.java

```java
package shop.model;

public final class Address {
    private final String street;
    private final String city;
    private final String postalCode;

    public Address(String street, String city, String postalCode) {
        if (street == null || street.isBlank()) {
            throw new IllegalArgumentException("Street must not be blank");
        }

        if (city == null || city.isBlank()) {
            throw new IllegalArgumentException("City must not be blank");
        }

        if (postalCode == null || postalCode.isBlank()) {
            throw new IllegalArgumentException("Postal code must not be blank");
        }

        this.street = street;
        this.city = city;
        this.postalCode = postalCode;
    }

    public String getString() {
        return street;
    }
```

```java
    public String getCity() {
        return city;
    }

    public String getPostalCode() {
        return postalCode;
    }

    @Override
    public String toString() {
        return street + ", " + city + " " + postalCode;
    }
}
```

Gambar 17 Kode program Class Address.java

```java
package shop.model;

public final class Address {
    private final String street;
    private final String city;
    private final String postalCode;
```

Fungsi class untuk menyimpan data alamat pengiriman (jalan, kota, kode pos). final class artinya class ini tidak bisa diwarisi. Semua field private final, jadi Address bersifat immutable yaitu setelah dibuat, isinya tidak bisa diubah.

```
public Address(String street, String city, String postalCode) {
    if (street == null || street.isBlank()) {
        throw new IllegalArgumentException("Street must not be blank");
    }

    if (city == null || city.isBlank()) {
        throw new IllegalArgumentException("City must not be blank");
    }

    if (postalCode == null || postalCode.isBlank()) {
        throw new IllegalArgumentException("Postal code must not be blank");
    }

    this.street = street;
    this.city = city;
    this.postalCode = postalCode;
```

Street bertipe string untuk nama jalan dan nomor (tidak boleh null), city bertipe string untuk nama kota (tidak boleh null), dan postalCode bertipe string untuk kode pos (tidak boleh null). Kode pos bertipe String (bukan angka) supaya angka nol di depan tidak hilang. Jika ada yang tidak valid, dilempar IllegalArgumentException.

```
public String getString() {
    return street;
```

Berfungsi untuk mengambil dan mengembalikan nama jalan.

```
public String getCity() {
    return city;
}

public String getPostalCode() {
    return postalCode;
}

@Override
public String toString() {
    return street + ", " + city + " " + postalCode;
}
```

getCity() dan getPostalCode(): getter biasa untuk kota dan kode pos. Keduanya mengembalikan String.

toString() untuk mengubah objek alamat menjadi teks. @Override karena toString() aslinya milik class Object.

18. Class Order.java

```java
package shop.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Order {
    private final String orderId;
    private final Address shippingAddress;
    private final List<Line> lines = new ArrayList<>();

    public Order(String orderId, Address shippingAddress) {
        if (orderId == null || orderId.isBlank() || shippingAddress == null) {
            throw new IllegalArgumentException("Order id and address are required");
        }

        this.orderId = orderId;
        this.shippingAddress = shippingAddress;
    }

    public String getOrderId() {
        return orderId;
    }

    public Address getShippingAddress() {
        return shippingAddress;
    }

    public void addLine(Product product, int quantity) {
        lines.add(new Line(product, quantity));
    }

    public List<Line> getLines() {
        return Collections.unmodifiableList(lines);
    }
```

```java
    public static class Line {
        private final Product product;
        private final int quantity;

        public Line(Product product, int quantity) {
            if (product == null || quantity <= 0) {
                throw new IllegalArgumentException("Product and quantity are required");
            }

            this.product = product;
            this.quantity = quantity;
        }

        public Product getProduct() {
            return product;
        }

        public int getQuantity() {
            return quantity;
        }

        public long subtotal() {
            return product.getFinalPrice() * quantity;
        }
    }

    public long getItemsTotal() {
        long total = 0;

        for (Line line : lines) {
            total += line.subtotal();
        }

        return total;
    }
```

```
    public class Receipt {
        public String toText() {
            StringBuilder result = new StringBuilder();

            result.append("STRUK PESANAN ").append(Order.this.orderId);

            for (Line line : lines) {
                result.append("\n- ")
                      .append(line.getProduct().getName())
                      .append(" x")
                      .append(line.getQuantity())
                      .append(" = Rp")
                      .append(line.subtotal());
            }

            result.append("\nTotal pesanan: Rp").append(Order.this.getItemsTotal());
            result.append("\nKirim ke: ").append(Order.this.shippingAddress);

            return result.toString();
        }
    }

    public Receipt createReceipt() {
        return new Receipt();
    }
}
```

Gambar 18 Kode program Class Order.java

```
package shop.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Order {
    private final String orderId;
    private final Address shippingAddress;
    private final List<Line> lines = new ArrayList<>();
```

Berfungsi untuk merepresentasikan satu pesanan: nomor pesanan, alamat tujuan, dan daftar barang yang dipesan. orderId untuk nomor pesanan, shippingAddress untuk alamat tujuan (objek Address), dan lines untuk daftar baris pesanan (Line). Langsung dibuat di deklarasi dengan new ArrayList<>(). Daftar ini final artinya variabelnya tidak bisa diganti, tetapi isinya masih bisa ditambah lewat addLine().

```
public Order(String orderId, Address shippingAddress) {
    if (orderId == null || orderId.isBlank() || shippingAddress == null) {
        throw new IllegalArgumentException("Order id and address are required");
    }

    this.orderId = orderId;
    this.shippingAddress = shippingAddress;
}
```

orderId dan shippingAddress tidak boleh null. Kedua validasi digabung dalam satu if dengan ||, jadi satu pesan error dipakai untuk keduanya.

```
public String getOrderId() {
    return orderId;
}

public Address getShippingAddress() {
    return shippingAddress;
}
```

Getter untuk nomor pesanan (String) dan alamat (Address).

```
public void addLine(Product product, int quantity) {
    lines.add(new Line(product, quantity));
}

public List<Line> getLines() {
    return Collections.unmodifiableList(lines);
}
```

addLine(Product product, int quantity)

Berfungsi untuk menambah satu baris pesanan (produk beserta jumlahnya) dengan parameter product dan quantity. Method ini membuat objek Line baru. Validasi produk dan jumlah dilakukan di constructor Line.

getLines()

berfungsi untuk mengambil daftar baris pesanan.

Collections.unmodifiableList(...) membungkus daftar agar pemanggil tidak bisa menambah atau menghapus isinya (add() akan melempar UnsupportedOperationException).

```
public Line(Product product, int quantity) {
    if (product == null || quantity <= 0) {
        throw new IllegalArgumentException("Product and quantity are required");
    }

    this.product = product;
    this.quantity = quantity;
}
```

Method Line dengan parameter product dan quantity. Product == 0 dan quantity <= 0.

```
public Product getProduct() {
    return product;
}

public int getQuantity() {
    return quantity;
}

public long subtotal() {
    return product.getFinalPrice() * quantity;
}
```

Mengambil dan mengembalikan nilai product dan quantity. Subtotal() mengembalikan product.getFinalPrice() * quantity.

```
public long getItemsTotal() {
    long total = 0;

    for (Line line : lines) {
        total += line.subtotal();
    }

    return total;
```

Berfungsi untuk menghitung total harga semua baris pesanan (belum termasuk ongkos kirim). Tiap Line dijumlahkan lewat subtotal().

```
public class Receipt {
    public String toText() {
        StringBuilder result = new StringBuilder();

        result.append("STRUK PESANAN ").append(Order.this.orderId);
```

```
for (Line line : lines) {
    result.append("\n- ")
          .append(line.getProduct().getName())
          .append(" x")
          .append(line.getQuantity())
          .append(" = Rp")
          .append(line.subtotal());
}

result.append("\nTotal pesanan: Rp").append(Order.this.getItemsTotal());
result.append("\nKirim ke: ").append(Order.this.shippingAddress);

return result.toString();
```

Berfungsi untuk membuat struk pesanan dalam bentuk teks. Receipt tidak static, jadi ia adalah inner class. Setiap objek Receipt selalu terikat ke satu objek Order, dan bisa mengakses field Order (termasuk yang private). Karena itulah ShopApp menulis order.new Receipt().

```
for (Line line : lines) {
    result.append("\n- ")
          .append(line.getProduct().getName())
          .append(" x")
          .append(line.getQuantity())
          .append(" = Rp")
          .append(line.subtotal());
}

result.append("\nTotal pesanan: Rp").append(Order.this.getItemsTotal());
result.append("\nKirim ke: ").append(Order.this.shippingAddress);

return result.toString();
```

Berfungsi untuk membuat struk pesanan dalam bentuk teks. Receipt tidak static, jadi ia adalah inner class. Setiap objek Receipt selalu terikat ke satu objek Order, dan bisa mengakses field Order (termasuk yang private). Karena itulah ShopApp menulis order.new Receipt().StringBuilder dipakai untuk menyusun teks panjang secara efisien, karena menggabung String dengan + berulang kali di dalam loop membuat banyak objek sementara.

Order.this.orderId artinya “field orderId milik objek Order yang membungkus Receipt ini”. Awalan Order.this dipakai untuk merujuk ke objek luar. Tulis judul dengan nomor pesanan, ulangi untuk tiap baris (nama, jumlah, subtotal), lalu tulis total pesanan dan alamat tujuan.

SelfCheck Latihan

```
=== SELF CHECK: Praktikum Abstraction ===


-- L1 - Abstract class & abstract method
  [LULUS] L1.1 Product adalah abstract class
  [LULUS] L1.2 Product punya abstract method category()
  [LULUS] L1.3 PhysicalProduct & DigitalProduct adalah subclass konkret Product
  [LULUS] L1.4 Konstruktor menolak harga negatif (IllegalArgumentException)
  [LULUS] L1.5 getInfo() PhysicalProduct sesuai format
  [LULUS] L1.6 ShippingService abstract dengan abstract method ratePerKg()
  [LULUS] L1.7 calculateCost(): Reguler 700g=10000, 1001g=20000, 0g=0; Ekspres 700g=18000

-- L2 - Interface & program to an interface
  [LULUS] L2.1 Shippable interface; PhysicalProduct implements, DigitalProduct tidak
  [LULUS] L2.2 PaymentMethod interface dengan getName() dan pay(long)
  [LULUS] L2.3 EWalletPayment: bayar sukses mengurangi saldo, saldo kurang ditolak
  [LULUS] L2.4 CheckoutService.checkout(...) menerima PaymentMethod & ShippingService (bukan class konkret)
  [LULUS] L2.5 Total keranjang contoh: Reguler=260000, Ekspres=268000

-- L3 - Default, static, private method; interface inheritance
  [LULUS] L3.1 Discountable: applyDiscount default, percentOf static, ada private method
  [LULUS] L3.2 DigitalProduct implements Discountable; harga akhir 50000 -> 45000
  [LULUS] L3.3 RefundablePayment & InstallmentPayment adalah interface turunan PaymentMethod
  [LULUS] L3.4 CreditCardPayment implements RefundablePayment DAN InstallmentPayment
  [LULUS] L3.5 monthlyInstallment(100000, 3)=33334 dan 13 bulan ditolak
  [LULUS] L3.6 CheckoutService.refund(...) hanya menerima RefundablePayment

-- L4 - final method & final class
  [LULUS] L4.1 ShippingService.calculateCost(int) adalah final method
  [LULUS] L4.2 Address final class, semua field private final, tanpa setter

-- L5 - Static nested, inner, dan anonymous class
  [LULUS] L5.1 Order.Line static nested; Order.Receipt inner (non-static)
  [LULUS] L5.2 Receipt.toText() memuat id pesanan, total barang, dan kota tujuan
  [LULUS] L5.3 ShopApp memakai anonymous class yang mengimplementasikan PaymentMethod

=== HASIL: 23 / 23 pemeriksaan lulus ===
```

Hasil ShopApp

```
== Katalog ==
[Fisik] Kopi Arabika Gayo 250g - Rp85000
[Fisik] Rendang Kemasan 250g - Rp120000
[Digital] E-book Resep Kue Nusantara - Rp45000

== Ringkasan ==
Subtotal : Rp250000
Berat kiriman : 700 g
Total Reguler : Rp260000
Total Ekspres : Rp268000

== Pembayaran ==
E-Wallet DompetKu berhasil? true
Sisa saldo: Rp40000
Transfer Bank Nagari berhasil? true
Kartu Kredit (Siti Rahma) berhasil? true
Cicilan 3 bulan: Rp89334

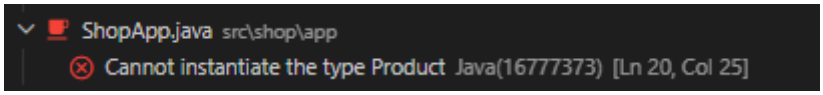
== Refund e-book ==
Refund e-wallet berhasil? true
Saldo setelah refund: Rp85000

STRUK PESANAN ORD-0001
- Kopi Arabika Gayo 250g x2 = Rp170000
- E-book Resep Kue Nusantara x1 = Rp45000
Total pesanan: Rp215000
Kirim ke: Jl. Moh Hatta No. 10, Padang 25121

Voucher Promo Hari Ini untuk e-book berhasil? true
```

Compiler Error Latihan

L1 – ShopApp



Muncul error karena Product adalah abstract class dan tidak bisa dibuat objek secara langsung.

L3 – ShopApp

checkout.refund(new BankTransferPayment("Bank Nagari"), 1_000);

compile error: BankTransferPayment is not RefundablePayment

karena BankTransferPayment bukan turunan atau implementasi dari RfundablePayment, sedangkan method checkout.refund() membutuhkan objek RefundablePayment.

L4 – FreeShipping

```
package shop.shipping;

public class FreeShipping extends ShippingService {

    public FreeShipping() {
        super(name: "Gratis");
    }

    @Override
    protected long ratePerKg() {
        return 0;
    }

    @Override
    public long calculateCost(int weightGrams) {
        return 0;
    }
```



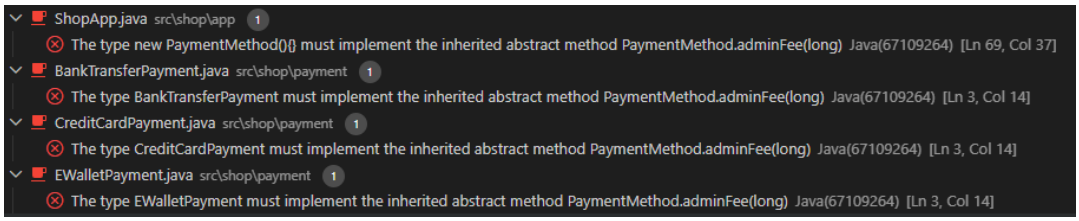
Method calculateCost() tidak boleh di override karena sudah diberi keyword final.

TUGAS PRAKTIKUM

NimParams.java

```
Parameter tugas untuk NIM 2511531018
  CARD_ADMIN_PERCENT  = 3
  TRANSFER_ADMIN_FEE  = 4000
  SMS_MAX_LENGTH      = 118
```

T1.a Menambahkan method abstract long adminFee(long amount); ke PaymentMethod



Terdapat 4 error terdiri dari:

- CreditCardPayment.java
- BankTransferPayment.java
- EwalletPayment.java
- ShopApp.java

Terjadi 4 error karena semua class yang mengimplementasikan PaymentMethod wajib menyediakan method adminFee(). Anonymous class voucher juga ikut error karena menggunakan interface PaymentMethod dan belum menambahkan method baru.

T1.b Menambahkan default method

```java
public interface PaymentMethod {
    String getName();

    boolean pay(long amount);

    default long adminFee(long amount){
    return 0;
    }
```

Semua error hilang tanpa perlu mengubah class implementasi. Default menyediakan implementasi bawaan sehingga class yang belum memiliki override otomatis menggunakan nilai 0.

T1.c Override sesuai kebutuhan

Class CreditCardPayment.java

```
@Override
public long adminFee(long amount) {
    return amount * StudentInfo.CARD_ADMIN_PERCENT / 100;
}
```

Biaya admin dihitung berdasarkan presentase dari jumlah pembayaran.

Class BankTransferPayment.java

```
@Override
public long adminFee(long amount) {
    return StudentInfo.TRANSFER_ADMIN_FEE;
}
```

Biaya admin menggunakan nominal tetap dari StudentInfo.

Class EwalletPayment.java

Tidak perlu diubah karena otomatis menggunakan default method yang mengembalikan nilai 0.

T1.d CheckoutService.java

```
public long grandTotal(List<Product> items, ShippingService shipping,
    PaymentMethod payment) {
    long total = total(items, shipping);

    return total + payment.adminFee(total);
}
```

Total(items, shipping) untuk menghitung total belanja beserta ongkir, payment.adminFee(total) menghitungb biaya admin sesuai metode pembayaran, dan keduanya dijumlahkan untuk mendapatkan total akhir.

Ubah method checkout

```
public boolean checkout(List<Product> items, ShippingService shipping, PaymentMethod payment) {
    if (items == null || items.isEmpty()) {
        return false;
    }
    return payment.pay(grandTotal(items, shipping, payment));
}
```

Mengubah perhitungan total pembayaran, jadi checkout menggunakan total belanja yang sudah ditambah biaya admin.

T1.e Konflik diamond

Menambahkan default method yang sama ke dua interface

Interface RefunadablePayment.java

```
default String policy() {
    return "Return maksimal 7 hari";
}
```

Interface InstallmentPayment.java

```
default String policy() {
    return "Cicilan maksimal " + maxInstallmentMonths() + " bulan";
}
```

Error pada class CreditCardPayment.java

Duplicate default methods named policy with the parameters () and () are inherited from the types InstallmentPayment and RefundablePayment.

Menambahkan Override

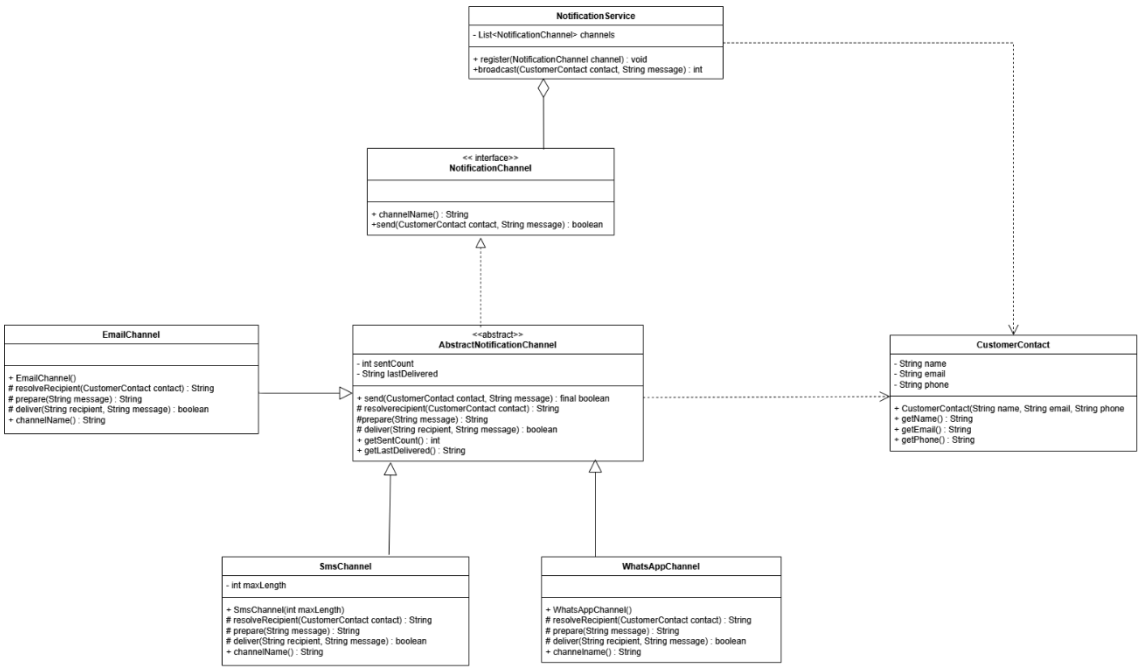
```
@Override
public String policy() {
    return RefundablePayment.super.policy() + "; " +
    InstallmentPayment.super.policy();
}
```

Hasilnya akan menggabungkan kedua kebijakan.

1) EwalletPayment tidak mengimplementasikan interface RefunablePayment dan InstallmentPayment secara bersamaan. Jadi, tidak terjadi konflik default method pada class tersebut.
2) Kedua interface memiliki method dengan nama dan parameter yang sama, tetapi implementasinya berbeda. Java tidak bisa menentukan kebijakan mana yang harus digunakan sehingga programmer harus meng-override method tersebut secara manual.

T2 Merancang sistem notifikasi pesanan

Langkah 1 – Rancang UML



Langkah 2 – Implementasi di package shop.notification.

1. CustomerContact

```java
package shop.notification;

public final class CustomerContact {
    private final String name;
    private final String email;
    private final String phone;

    public CustomerContact(String name, String email, String phone) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }

        this.name = name;
        this.email = email == null ? "" : email;
        this.phone = phone == null ? "" : phone;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getPhone() {
        return phone;
    }
```

```java
public final class CustomerContact {
    private final String name;
    private final String email;
    private final String phone;
```

Kata final di class berarti class ini tidak bisa di-extend (diwarisi). Tidak ada yang bisa membuat class Lain extends CustomerContact. Private artinya hanya bisa diakses dari dalam class ini. Class lain harus lewat getter. Final artinya nilainya hanya bisa diisi sekali (di constructor) dan tidak bisa diganti lagi. Gabungan keduanya membuat objek ini immutable (tidak bisa berubah setelah dibuat).

```java
public CustomerContact(String name, String email, String phone) {
    if (name == null || name.isBlank()) {
        throw new IllegalArgumentException("Name must not be blank");
    }

    this.name = name;
    this.email = email == null ? "" : email;
    this.phone = phone == null ? "" : phone;
}
```

Nama wajib ada. Kalau null atau cuma berisi spasi/kosong, constructor melempar error dan objek tidak jadi dibuat. Jadi new CustomerContact(null, ...) akan langsung gagal. Artinya email dan telepon boleh kosong, getEmail() dan getPhone() tidak pernah mengembalikan null.

```java
public String getName() {
    return name;
}

public String getEmail() {
    return email;
}

public String getPhone() {
    return phone;
}
```

Cuma ada getter, tidak ada setter, konsisten dengan sifat immutable tadi: data bisa dibaca, tapi tidak bisa diubah dari luar.

2. NotificationChannel

```java
package shop.notification;

public interface NotificationChannel {

    String channelName();
    boolean send(CustomerContact contact, String message);

}
```

Interface ini hanya berisi “siapapun pun yang mau jadi kanal notifikasi harus punya channelName() dan send()”. Tidak ada isi logikanya. Karena itu NotificationService cukup tahu tipe NotificationChannel, tanpa peduli kanalnya Email, SMS, WhatsApp, atau LOG.

3. AbstractChannel

```java
package shop.notification;

public abstract class AbstractChannel implements NotificationChannel {
    private int sentCount = 0;
    private String lastDelivered = "";
    protected abstract String resolveRecipient(CustomerContact contact);
    protected abstract String prepare(String message);
    protected abstract boolean deliver(String recipient, String message);

    @Override
    public final boolean send(CustomerContact contact, String message) {
        if (contact == null || message == null || message.isBlank()) {
            return false;
        }
        String recipient = resolveRecipient(contact);
        if (recipient == null || recipient.isBlank()) {
            return false;
        }
        String preparedMessage = prepare(message);
        if (deliver(recipient, preparedMessage)) {
            sentCount++;
            lastDelivered = preparedMessage;
            return true;
        }
        return false;
    }

    public int getSentCount() {
        return sentCount;
    }
    public String getLastDelivered() {
        return lastDelivered;
    }
```

```java
public abstract class AbstractChannel implements NotificationChannel {
    private int sentCount = 0;
    private String lastDelivered = "";
    protected abstract String resolveRecipient(CustomerContact contact);
    protected abstract String prepare(String message);
    protected abstract boolean deliver(String recipient, String message);
```

abstract artinya class ini tidak bisa di-new langsung, hanya bisa diwarisi. implements NotificationChannel membuat semua turunannya (Email, Sms, WhatsApp) otomatis bertipe NotificationChannel. Pencatat statistik kanal yaitu berapa kali sukses kirim dan pesan terakhir yang terkirim. Keduanya private, jadi hanya bisa berubah lewat send().resolveRecipient menentukan tujuan (email atau nomor HP) dan mengembalikan null kalau tidak valid. prepare menyesuaikan pesan dengan aturan kanal, misalnya dipotong untuk SMS. deliver melakukan pengiriman dan mengembalikan true kalau sukses.

```
@Override
public final boolean send(CustomerContact contact, String message) {
    if (contact == null || message == null || message.isBlank()) {
        return false;
    }
    String recipient = resolveRecipient(contact);
    if (recipient == null || recipient.isBlank()) {
        return false;
    }
    String preparedMessage = prepare(message);
    if (deliver(recipient, preparedMessage)) {
        sentCount++;
        lastDelivered = preparedMessage;
        return true;
    }
    return false;
```

Ini alur utamanya, dan final membuatnya tidak bisa ditimpa subclass. Urutannya yaitu validasi input, cari tujuan (kalau null, kanal dilewati), siapkan pesan, lalu kirim. Kalau deliver sukses, statistik diperbarui dan hasilnya true, kalau tidak hasilnya false.

```
public int getSentCount() {
    return sentCount;
}
public String getLastDelivered() {
    return lastDelivered;
```

Satu-satunya cara membaca statistik dari luar, karena fieldnya private dan tidak ada setter.

4. EmailChannel

```java
package shop.notification;

public class EmailChannel extends AbstractChannel {
    public EmailChannel() {
    }

    @Override
    public String channelName() {
        return "Email";
    }

    @Override
    protected String resolveRecipient(CustomerContact contact) {
        String email = contact.getEmail();

        if (email.contains("@")) {
            return email;
        }
        return null;
    }
    @Override
    protected String prepare(String message) {
        return message;
    }
    @Override
    protected boolean deliver(String recipient, String message) {
        System.out.println("Email dikirim ke " + recipient);
        System.out.println("Pesan: " + message);
        return true;
    }
```

```java
public class EmailChannel extends AbstractChannel {
    public EmailChannel() {
    }

    @Override
    public String channelName() {
        return "Email";
    }
```

extends AbstractChannel membuat EmailChannel mewarisi method send(), getSentCount(), dan getLastDelivered(). Karena AbstractChannel meng-implement NotificationChannel, EmailChannel otomatis juga bertipe NotificationChannel. Memenuhi kontrak dari interface NotificationChannel. Method ini wajib ditulis di sini karena AbstractChannel tidak mengisinya. Hasilnya dipakai sebagai nama kanal.

```java
    @Override
    protected String resolveRecipient(CustomerContact contact) {
        String email = contact.getEmail();

        if (email.contains("@")) {
            return email;
        }
        return null;
    }
```

Mengambil email dari kontak. Kalau mengandung @, email dianggap valid dan dikembalikan sebagai tujuan. Kalau tidak, mengembalikan null, sehingga send() di AbstractChannel menghentikan proses dan kanal ini dilewati. Tidak ada risiko NullPointerException karena CustomerContact sudah mengubah email null menjadi "".

```
@Override
protected String prepare(String message) {
    return message;
}
@Override
protected boolean deliver(String recipient, String message) {
    System.out.println("Email dikirim ke " + recipient);
    System.out.println("Pesan: " + message);
    return true;
}
```

Pesan dikembalikan apa adanya, karena email tidak punya batas panjang atau format khusus seperti SMS dan WhatsApp. Method ini tetap harus ada karena sifatnya abstrak di AbstractChannel. Simulasi pengiriman yaitu mencetak tujuan dan isi pesan, lalu mengembalikan true sebagai tanda sukses.

5. SmsChannel

```
package shop.notification;

public class SmsChannel extends AbstractChannel {
    private final int maxLength;

    public SmsChannel(int maxLength) {
        if (maxLength < 3) {
            throw new IllegalArgumentException("maxLength minimal 3");
        }
        this.maxLength = maxLength;
    }

    @Override public String channelName() {
        return "SMS";
    }

    @Override
    protected String resolveRecipient(CustomerContact contact) {
        String phone = contact.getPhone();

        if (phone.startsWith("08")) {
            return phone;
        }

        return null;
    }
```

```
    @Override
    protected String prepare(String message) {
        if (message.length() > maxLength) {
            return message.substring(0, maxLength - 3) + "...";
        }
        return message;
    }
    @Override
    protected boolean deliver(String recipient, String message) {
        System.out.println("SMS dikirim ke " + recipient);
        System.out.println("Pesan: " + message);
        return true;
    }
```

```
public class SmsChannel extends AbstractChannel {
    private final int maxLength;

    public SmsChannel(int maxLength) {
        if (maxLength < 3) {
            throw new IllegalArgumentException("maxLength minimal 3");
        }
        this.maxLength = maxLength;
```

extends AbstractChannel membuat SmsChannel mewarisi send() dan dua getter statistik, sekaligus otomatis bertipe NotificationChannel. Field maxLength menyimpan batas panjang pesan SMS, dibuat private final sehingga hanya diisi sekali di constructor. Constructor memvalidasi bahwa maxLength minimal 3.

```
@Override public String channelName() {
    return "SMS";
}

@Override
protected String resolveRecipient(CustomerContact contact) {
    String phone = contact.getPhone();

    if (phone.startsWith("08")) {
        return phone;
    }

    return null;
```

Memenuhi kontrak interface. Cara penulisannya (@Override satu baris dengan public) hanya soal gaya, fungsinya sama. Mengambil nomor HP dari kontak. Kalau diawali 08, nomor dikembalikan sebagai tujuan. Kalau tidak, mengembalikan null dan kanal ini dilewati oleh send(). Aman dari NullPointerException karena CustomerContact mengubah null menjadi "".

```
    @Override
    protected String prepare(String message) {
        if (message.length() > maxLength) {
            return message.substring(0, maxLength - 3) + "...";
        }
        return message;
    }
    @Override
    protected boolean deliver(String recipient, String message) {
        System.out.println("SMS dikirim ke " + recipient);
        System.out.println("Pesan: " + message);
        return true;
    }
```

Kalau pesan lebih panjang dari maxLength, pesan dipotong sampai maxLength - 3 karakter lalu ditambah "...". Jadi total panjangnya persis maxLength. Kalau pesan masih muat, dikembalikan apa adanya. Simulasi

pengiriman yaitu mencetak tujuan dan pesan yang sudah dipotong, lalu mengembalikan true.

6. WhatsAppChannel

```java
package shop.notification;

public class WhatsAppChannel extends AbstractChannel {

    public WhatsAppChannel() {
    }

    @Override
    public String channelName() {
        return "WhatsApp";
    }

    @Override
    protected String resolveRecipient(CustomerContact contact) {
        String phone = contact.getPhone();

        if (phone.length() >= 10 && (phone.startsWith("08") || phone.startsWith("+62"))) {
            return phone;
        }

        return null;
    }

    @Override
    protected String prepare(String message) {
        return "[Toko UMKM] " + message;
    }

    @Override
    protected boolean deliver(String recipient, String message) {
        System.out.println("WhatsApp dikirim ke " + recipient);
        System.out.println("Pesan: " + message);
        return true;
```

```java
public class WhatsAppChannel extends AbstractChannel {

    public WhatsAppChannel() {
    }

    @Override
    public String channelName() {
        return "WhatsApp";
    }
```

extends AbstractChannel membuat WhatsAppChannel mewarisi send() dan dua getter statistik, sekaligus otomatis bertipe NotificationChannel. Memenuhi kontrak interface NotificationChannel dengan mengembalikan nama kanal.

```java
@Override
protected String resolveRecipient(CustomerContact contact) {
    String phone = contact.getPhone();

    if (phone.length() >= 10 && (phone.startsWith("08") || phone.startsWith("+62"))) {
        return phone;
    }

    return null;
}
```

Mengambil nomor HP dari kontak, lalu memvalidasi dua hal sekaligus: panjangnya minimal 10 karakter, dan awalannya 08 atau +62. Kalau keduanya terpenuhi, nomor dikembalikan sebagai tujuan, kalau tidak mengembalikan null sehingga kanal ini dilewati. Perbedaannya dengan SmsChannel adalah WhatsApp juga menerima format internasional +62. Aman dari NullPointerException karena CustomerContact mengubah null menjadi "", dan "".length() hanya bernilai 0 sehingga ditolak.

```
@Override
protected String prepare(String message) {
    return "[Toko UMKM] " + message;
}

@Override
protected boolean deliver(String recipient, String message) {
    System.out.println("WhatsApp dikirim ke " + recipient);
    System.out.println("Pesan: " + message);
    return true;
```

Menambahkan awalan “[Toko UMKM] “ di depan pesan sebagai identitas toko. Tidak ada pemotongan seperti SMS, jadi pesan panjang tetap utuh. Simulasi pengiriman yaitu mencetak tujuan dan pesan, lalu mengembalikan true sebagai tanda sukses.

7. NotificationService

```
package shop.notification;

import java.util.ArrayList;
import java.util.List;

public class NotificationService {
    private final List<NotificationChannel> channels = new ArrayList<>();

    public void register(NotificationChannel channel) {
        if (channel != null) {
            channels.add(channel);
        }
    }
    public int broadcast(CustomerContact contact, String message) {
        int success = 0;

        for (NotificationChannel channel : channels) {
            if (channel.send(contact, message)) {
                success++;
            }
        }
        return success;
    }
```

```
import java.util.ArrayList;
import java.util.List;

public class NotificationService {
    private final List<NotificationChannel> channels = new ArrayList<>();
```

Class ini adalah “pengelola” semua kanal. Daftar channels bertipe List<NotificationChannel>, bukan List<EmailChannel> atau sejenisnya. Kata final berarti variabel channels tidak bisa diganti dengan list lain, tapi isinya tetap boleh bertambah lewat add. Karena private, daftar ini hanya bisa diubah lewat register().

```
public void register(NotificationChannel channel) {
    if (channel != null) {
        channels.add(channel);
    }
}
```

Method ini mendaftarkan kanal baru ke daftar. Kalau argumennya null, diabaikan supaya tidak ada elemen null di dalam list (yang nanti bikin NullPointerException di broadcast). Parameternya bertipe NotificationChannel, itulah sebabnya di TugasApp baris service.register(new EmailChannel()) hanya valid kalau EmailChannel benar-benar turunan dari NotificationChannel

```
public int broadcast(CustomerContact contact, String message) {
    int success = 0;

    for (NotificationChannel channel : channels) {
        if (channel.send(contact, message)) {
            success++;
        }
    }
    return success;
```

Method ini mengirim satu pesan lewat semua kanal yang terdaftar. Variabel success dimulai dari 0. Lalu untuk setiap kanal, send() dipanggil, dan kalau hasilnya true maka success bertambah. Kanal yang mengembalikan false (misalnya karena tujuannya tidak valid) tidak menghentikan loop, jadi kanal berikutnya tetap dicoba.

Langkah 3 – Demo

TugasApp.java

```java
package shop.app;

import java.util.List;

import shop.StudentInfo;
import shop.model.Product;
import shop.model.PhysicalProduct;
import shop.model.DigitalProduct;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.payment.*;

import shop.notification.*;

public class TugasApp {

    public static void main(String[] args) {

        PhysicalProduct coffee = new PhysicalProduct(sku: "KOPI-001", name: "Kopi", price: 85000, weightGrams: 300);
        DigitalProduct ebook = new DigitalProduct(sku: "EBK-003", name: "Ebook", price: 50000, downloadUrl: "https://example.com/ebook");

        List<Product> items = List.of(coffee, ebook);

        Run main | Debug main | Run | Debug
        CheckoutService checkout = new CheckoutService();
        ExpressShipping shipping = new ExpressShipping();

        EWalletPayment ewallet = new EWalletPayment(provider: "DompetKu", balance: 300000);
        BankTransferPayment transfer = new BankTransferPayment(bankName: "Bank Nagari");
        CreditCardPayment card = new CreditCardPayment(cardHolder: "Siti Rahma", creditLimit: 1000000);
```

```java
System.out.println("== T1: Total dengan biaya admin (Ekspres) ==");
System.out.println("E-Wallet DompetKu : Rp" + checkout.grandTotal(items, shipping, ewallet));
System.out.println("Transfer Bank Nagari : Rp" + checkout.grandTotal(items, shipping, transfer));
System.out.println("Kartu Kredit (Siti Rahma) : Rp" + checkout.grandTotal(items, shipping, card));

System.out.println();

System.out.println("Kebijakan kartu kredit : " + card.policy());
System.out.println("Kebijakan e-wallet : " + ewallet.policy());

System.out.println("\n== T2: Notifikasi status pesanan ==");

NotificationService service = new NotificationService();

service.register(new EmailChannel());
service.register(new SmsChannel(StudentInfo.SMS_MAX_LENGTH));
service.register(new WhatsAppChannel());

service.register(new NotificationChannel() {

    @Override
    public String channelName() {
        return "LOG";
    }

    @Override
    public boolean send(CustomerContact contact, String message) {
        System.out.println("[LOG] " + contact.getName()
            + ": " + message);
        return true;
    }
});
```

```
CustomerContact contact1 = new CustomerContact(
    name: "Siti",
    email: "siti@example.com",
    phone: "081234567890"
);

CustomerContact contact2 = new CustomerContact(
    name: "Budi",
    email: null,
    phone: "081234567891"
);

CustomerContact contact3 = new CustomerContact(
    name: "Ani",
    email: null,
    phone: "+6281234567892"
);

String message = "Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. "
    + "Nomor resi akan muncul di aplikasi dalam 1x24 jam. "
    + "Terima kasih telah berbelanja produk UMKM Sumatera Barat di toko kami!";

System.out.println("\nKontak pertama:");
int result1 = service.broadcast(contact1, message);
System.out.println("Terkirim ke " + result1 + " kanal");

System.out.println("\nKontak kedua:");
int result2 = service.broadcast(contact2, message);
System.out.println("Terkirim ke " + result2 + " kanal");

System.out.println("\nKontak ketiga:");
int result3 = service.broadcast(contact3, message);
System.out.println("Terkirim ke " + result3 + " kanal");
```

Jawaban:

1. Mengapa ada interface dan abstract class, bukan salah satunya saja?

   Jawaban:

   Interface digunakan sebagai kontrak agar setiap kanal memiliki method yang sama. Abstract class digunakan untuk menyediakan proses pengiriman yang bisa digunakan bersama oleh semua kanal sehingga kode tidak perlu ditulis ulang.

2. Apa akibatnya jika send(...) tidak dibuat final?

   Jawaban:

   Jika tidak dibuat final, class turunannya dapat mengubah proses pengiriman notifikasi. Hal ini dapat menyebabkan alur pengiriman berbeda-beda pada setiap kanal.

3. Mengapa NotificationService menyimpan List<NotificationChannel>, bukan List<AbstractChannel>?

   Jawaban:

NotificationChannel lebih fleksibel. Dengan menggunakan interface, NotificationService bisa menerima class apapun yang mengimplementasikan interface tersebut, termasuk anonymous class dan kanal baru yang dibuat nantinya.

T3 – Review BadShop

5 kesalahan pada BadShop

No.	**Lokasi (class/method)**	**Masalah**	**Dampak nyata (bug/risiko)**	**Perbaikan yang diusulkan**
1.	StarOperation	Interface memiliki terlalu banyak tanggung jawab	Class yang tidak membutuhkan refund atau pengiriman tetap harus mengimple-mentasikan method tersebut.	Memisahkan interface berdasarkan tanggung jawab.
2.	GiftVoucher.refund() dan ship()	Method melempar Unsupported-OperationException	Program dapat berhenti ketika method dipanggil.	Hapus method yang tidak seuai dengan kemampuan voucher melalui pemisahan interface.
3.	Courier.cost()	Menggunakan string untuk	Salah ketik “EXPRES” menghasilkan	Gunakan enum ShippingType

		menentukan jenis pengiriman	ongkir 0 tanpa peringatan.	agar pilihan jenis pengiriman lebih terkontrol
4.	Courier	Class abstract tidak memiliki abstract method	Subclass tidak diwajibkan menyediakan implementasi perhitungan ongkir	Tambahkan abstract method untuk menentukan tarif pengiriman
5	Address	Atribut dapat diubah melalui setter	Data alamat pesanan bisa berubah setelah objek dibuat	jadikan atribut final dan hapus setter agar address immutable

Perbaikan 3 kesalahan

1. **StoreOperation**

```
// PERBAIKAN 1: Interface dipisahkan sesuai tanggung jawab
interface PaymentOperation {
    boolean pay(long amount);
}

interface InvoiceOperation {
    String printInvoice();
}

interface ShippingOperation {
    long ship(int weightGrams);
}

class GiftVoucher implements PaymentOperation, InvoiceOperation {
    private double balance = 100000.0;

    @Override
    public boolean pay(long amount) {
        if (amount > balance || amount <= 0) {
            return false;
        }
        balance -= amount;
        return true;
    }

    @Override
    public String printInvoice() {
        return "Voucher, remaining balance " + balance;
```

2. **Courier**

```java
// PERBAIKAN 2: Menggunakan enum untuk jenis pengiriman
enum ShippingType {
    REGULAR,
    EXPRESS,
    SAME_DAY
}

abstract class Courier {
    public long cost(ShippingType type, int kilograms) {
        switch (type) {
            case REGULAR:
                return kilograms * 10000L;
            case EXPRESS:
                return kilograms * 18000L;
            case SAME_DAY:
                return kilograms * 30000L;
            default:
                return 0;
        }
```

3. Address

```java
// PERBAIKAN 3: Address dibuat immutable
final class Address {
    private final String street;
    private final String city;

    Address(String street, String city) {
        this.street = street;
        this.city = city;
    }

    @Override
    public String toString() {
        return street + ", " + city;
    }
}
```

Hasil:

```
Total cart: 170000
Ongkir Ekspres 2 kg: 36000
ORD-1 -> Jl. Sudirman 10, Padang
ORD-2 -> Jl. Sudirman 10, Bukittinggi
Harga flash sale: 70000
true | Voucher, remaining balance 70000.0
```

T4 – Refleksi

Tabel 1 – Abstract class vs interface

Pertanyaan desain	Abstract class / interface	Alasan	Contoh dari Proyek
Subclass berbagi field dan konstruktor	Abstract class	Karena bisa menyimpan atribut	Product memiliki field sku, name, dan price yang digunakan

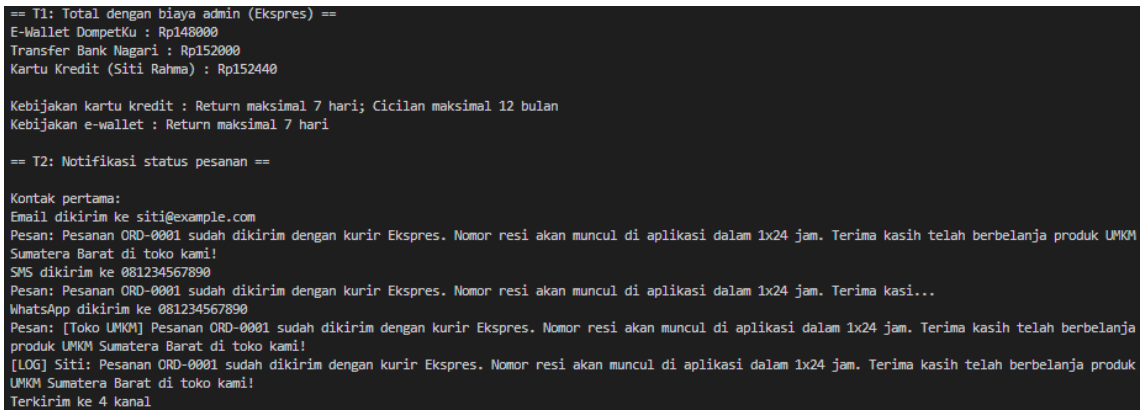
		dan konstruktor yang diwariskan subclass.	PhysicalProduct dan DigitalProduct.
Satu class perlu beberapa kemampuan sekaligus	Interface	Karena satu class bisa mengimplementasikan beberapa interface sekaligus.	PhysicalProduct mengimplementasikan Shippable dan DigitalProduct mengimplementasikan Discountable.
Rumus bersama yang tidak boleh diubah subclass	Abstract class	Karena bisa menggunakan method final agar implementasinya tidak bisa diubah subclass.	Method grandTotal() pada CheckoutService dapat dibuat final jika perhitungannya harus tetap sama.
Kontrak yang perlu ditambah method baru di kemudian hari	Interface	Karena interface bisa menggunakan default method untuk menambahkan kemampuan tanpa mewajibkan semua class mengubah implementasinya.	PaymentMethod memiliki default method adminFee().

Tabel 2 – Jenis nested class

Kondisi	Static nested, inner, atau anonymous?	Alasan	Contoh dari proyek
Class pembantu tidak membaca data objek luar	Static nested	Tidak membutuhkan	Cart.Item pada contoh project BadShop dapat

		objek dari class luar untuk dibuat.	dijadikan static nested class karena tidak menggunakan data Cart.
Objek harus membaca field objek luarnya	Inner	Karena membutuhkan akses ke atribut dan method milik objek luar.	Order.Receipt membaca informasi pesanan seperti ID dan daftar barang dari objek Order.
Implementasi sekali pakai, tidak perlu nama	Anonymous	Karena class dibuat langsung ketika dibutuhkan tanpa membuat file atau nama class baru.	Anonymous PaymentMethod untuk Voucher Promo Hari Ini di ShopApp.

TugasApp



```
Kontak kedua:
SMS dikirim ke 081234567891
Pesan: Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasi...
WhatsApp dikirim ke 081234567891
Pesan: [Toko UMKM] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Ter
produk UMKM Sumatera Barat di toko kami!
[LOG] Budi: Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kas
UMKM Sumatera Barat di toko kami!
Terkirim ke 3 kanal

Kontak ketiga:
WhatsApp dikirim ke +6281234567892
Pesan: [Toko UMKM] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Ter
produk UMKM Sumatera Barat di toko kami!
[LOG] Ani: Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasi
MKM Sumatera Barat di toko kami!
Terkirim ke 2 kanal
```

Tugas Check

```
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2511531018 | Syasya Halwa Gazwani

-- Parameter NIM
  [LULUS] P.1 Konstanta di StudentInfo sesuai NIM (3%, Rp4000, 118 karakter)

-- T1 - Evolusi API & diamond
  [LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
  [LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
  [LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
  [LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan

-- T2 - Sistem notifikasi
  [LULUS] T2.1 NotificationChannel interface; AbstractChannel abstract dengan send(...) final
  [LULUS] T2.2 EmailChannel, SmsChannel, WhatsAppChannel adalah subclass AbstractChannel
SMS dikirim ke 081234567890
Pesan: xxxx
```